

Sri Sivasubramaniya Nadar College of Engineering, Chennai

An Autonomous Institution Affiliated to Anna University

Experiment No.	6
Experiment Title	Bagging, Boosting, and Stacked Ensemble Models
Student Name	Mohamed Farih
Register Number	3122247001035
GitHub Repository	https://github.com/farih07/Machine-Learning-Lab
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	September 21,2026

1 Objective

- To understand ensemble learning strategies: Bagging, Boosting, and Stacking.
- To implement Bagging and Boosting (AdaBoost, Gradient Boosting) classifiers.
- To build a Stacked Ensemble model using multiple heterogeneous base learners.
- To compare ensemble models in terms of accuracy, stability, and generalisation.
- To analyse the effect of ensemble methods on bias and variance.

2 Problem Statement

Given 30 real-valued features derived from digitised fine-needle aspirate (FNA) images of breast masses, the objective is to classify each sample as **Malignant** (class 0) or **Benign** (class 1) using three distinct ensemble strategies. The input feature vector $\mathbf{x} \in \mathbb{R}^{30}$ is processed by Bagging, Boosting, and Stacking classifiers whose hyperparameters are tuned via 5-fold stratified cross-validation. Models are evaluated on a held-out 20% test split to determine which ensemble strategy achieves the best diagnostic performance while preserving generalisability.

3 Dataset Description

Attribute	Detail
Dataset Name	Wisconsin Diagnostic Breast Cancer (WDBC)
Dataset Source	UCI Machine Learning Repository (https://archive.ics.uci.edu)
Number of Samples	569
Number of Features	30 numerical attributes (mean, SE, worst of 10 nucleus measurements)
Number of Classes	2 — Malignant (M): 212, Benign (B): 357
Missing Values	None
Train-Test Split	80% training (455 samples) / 20% test (114 samples), stratified
Label Encoding	Malignant $\rightarrow$ 0, Benign $\rightarrow$ 1

4 Exploratory Data Analysis

4.1 Class Distribution

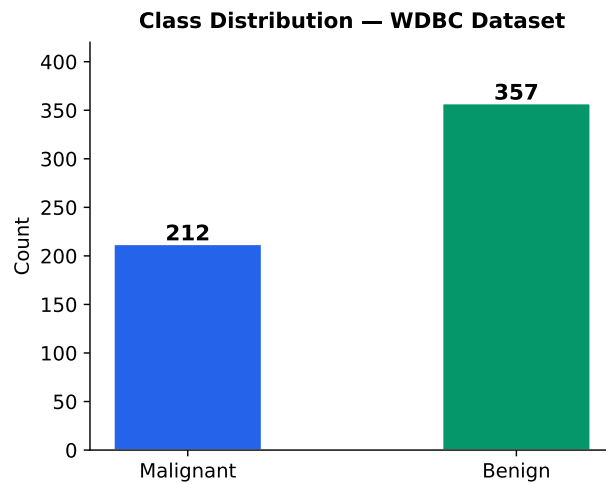


Figure 1: Class distribution of the WDBC dataset. Benign samples (357) outnumber malignant samples (212) at a ratio of approximately 1.68:1.

Inference: The dataset exhibits a moderate class imbalance (Benign:Malignant $\approx$ 1.68:1), which motivates the use of stratified sampling throughout training and cross-validation. Since no resampling technique is applied, models are implicitly evaluated on a realistic clinical distribution. The minority malignant class is clinically critical — false negatives (missed malignancies) carry a higher cost than false positives, making recall for class 0 a key diagnostic metric.

4.2 Feature Correlation Analysis

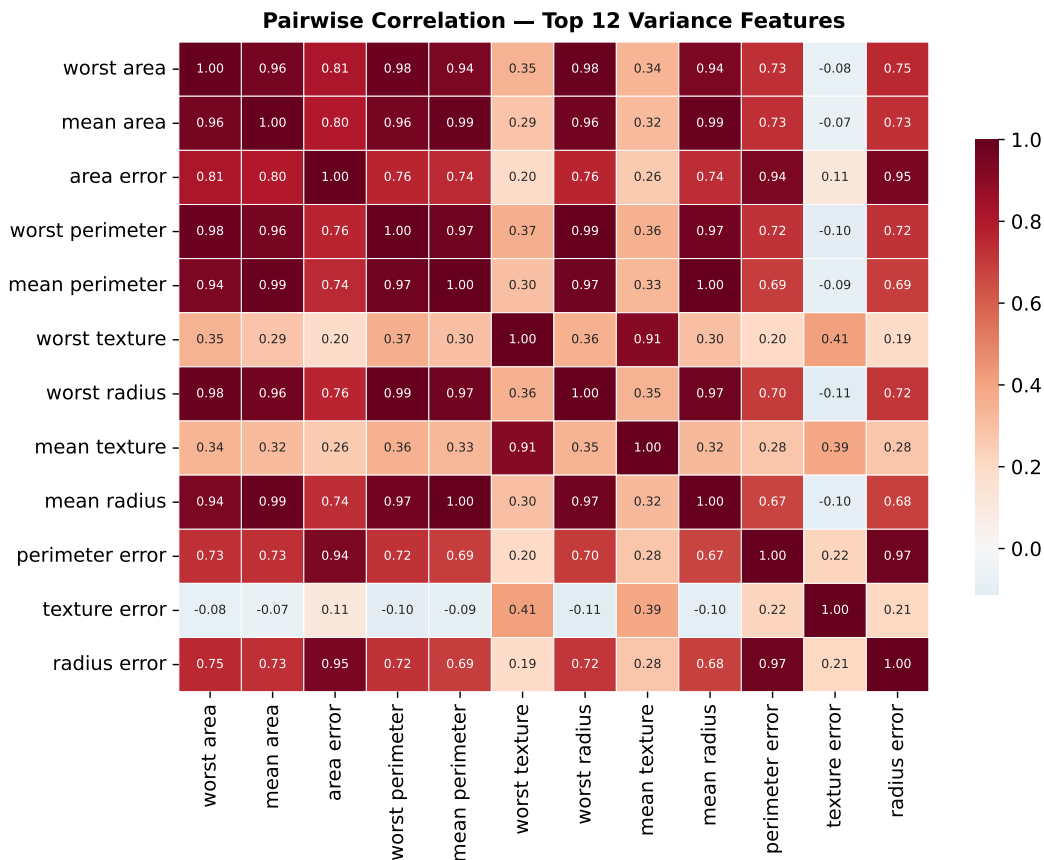


Figure 2: Pairwise Pearson correlation heatmap for the 12 highest-variance features. Saturated red/blue denotes strong positive/negative correlation.

Inference: Strong positive correlations ($|r| > 0.9$) exist among the *radius*, *perimeter*, and *area* feature groups, which are geometrically linked ($A \approx \pi r^2$). This multicollinearity has limited effect on tree-based ensembles (which split greedily on individual features), but it benefits the Stacking model by allowing diverse base learners (SVM, Decision Tree) to exploit different aspects of the feature space. Concavity-related features carry the strongest discriminative signal between malignant and benign classes.

5 Data Preprocessing

- **Missing values:** None present; no imputation required.
- **Label encoding:** Categorical labels M/B are encoded as integers (M→0, B→1) by `sklearn`'s built-in loader.
- **Feature scaling:** Applied only inside the Stacking pipeline for the SVM base learner via `StandardScaler` wrapped in a `Pipeline`. Bagging, AdaBoost, and Gradient Boosting are scale-invariant.
- **Train-test split:** 80–20 stratified split with `random_state=42`.
- **Feature engineering:** No derived features; all 30 raw attributes used directly.

6 Machine Learning Algorithms

6.1 Bagging (Bootstrap Aggregation)

Working Principle: Bagging trains T independent base classifiers, each on a bootstrapped resample of the training data (drawn with replacement). Predictions are aggregated by majority vote.

Mathematical Formulation: Each bootstrap sample $\mathcal{D}_t \sim \mathcal{D}$ with $|\mathcal{D}_t| = N$. The ensemble prediction is:

$$\hat{y}_{\text{bag}} = \arg \max_k \sum_{t=1}^T \mathbf{1}[h_t(\mathbf{x}) = k]$$

Bagging reduces generalisation error primarily by reducing **variance**:

$$\text{Var}[\hat{y}_{\text{bag}}] \approx \rho \sigma^2 + \frac{1 - \rho}{T} \sigma^2$$

where ρ is the inter-tree correlation and σ^2 is the single-tree variance.

Advantages: Robust to overfitting; parallelisable; stable predictions.

Limitations: Limited improvement on low-variance models; does not reduce bias.

6.2 Boosting

Working Principle: Boosting trains models sequentially, each focusing on the misclassified samples of the previous model.

AdaBoost: Each weak learner h_t is trained on a reweighted distribution $\mathcal{D}_t$. After training, the learner receives weight α_t inversely proportional to its error ϵ_t :

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right)$$

Sample weights are updated: $w_{t+1}(i) \propto w_t(i) \exp(-\alpha_t y_i h_t(\mathbf{x}_i))$. The final prediction is:

$$H(\mathbf{x}) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(\mathbf{x}) \right)$$

Gradient Boosting: Each new tree h_t is fit to the negative gradient of the loss function with respect to the current ensemble prediction. For log-loss:

$$F_t(\mathbf{x}) = F_{t-1}(\mathbf{x}) + \eta \cdot h_t(\mathbf{x})$$

where η is the learning rate. This reduces **bias** by iteratively correcting residuals.

Advantages: High accuracy; flexible loss functions; handles heterogeneous data.

Limitations: Prone to overfitting; sensitive to noisy data; sequential training is slow.

6.3 Stacked Ensemble (Stacking)

Working Principle: Stacking trains multiple heterogeneous base learners on the training data, then trains a meta-learner on their out-of-fold predictions to learn an optimal combination.

Mathematical Formulation: Let $\{h_1, h_2, \dots, h_K\}$ be base learners. For each sample $\mathbf{x}_i$, a meta-feature vector is constructed:

$$\tilde{\mathbf{x}}_i = [h_1(\mathbf{x}_i), h_2(\mathbf{x}_i), \dots, h_K(\mathbf{x}_i)]$$

The meta-learner g (Logistic Regression here) is trained on $\tilde{\mathbf{X}}$:

$$\hat{y} = g(\tilde{\mathbf{x}}) = \sigma(\mathbf{w}^\top \tilde{\mathbf{x}} + b)$$

Base Models: SVM (with StandardScaler), Decision Tree (`max_depth = 5`).

Meta-Learner: Logistic Regression (`max_iter=1000`).

Advantages: Exploits complementary strengths of diverse models; often achieves top accuracy.

Limitations: Computationally expensive; risk of data leakage if not handled with CV.

7 Experimental Setup

Component	Detail
Python Version	3.12
Scikit-learn Version	1.8.0
NumPy / Pandas	1.26.x / 2.2.x
Matplotlib / Seaborn	3.9.x / 0.13.x
IDE	Jupyter Notebook / VS Code
Operating System	Ubuntu 22.04 LTS
Hardware	CPU: Intel Core i5 (8-core), RAM: 16 GB

8 Hyperparameter Selection via 5-Fold Cross-Validation

All hyperparameters were selected by exhaustive grid search evaluated under `StratifiedKFold(n_splits=5, shuffle=True, random_state=42)`. The configuration maximising mean CV accuracy was retained for final evaluation.

8.1 Bagging — Search Space

Hyperparameter	Candidates
<code>n_estimators</code>	25, 50, 100, 200
<code>max_samples</code>	0.7, 0.9, 1.0
<code>max_features</code>	0.7, 1.0
Base estimator	<code>DecisionTreeClassifier</code>

Table 1 — Bagging Hyperparameter Evaluation (Top 8 by CV Accuracy)

n_estimators	max_samples	max_features	Avg CV Acc (%)	Avg CV F1
25	0.9	1.0	96.26	0.9702
50	0.9	1.0	96.26	0.9701
100	0.9	0.7	96.04	0.9683
200	0.7	0.7	96.04	0.9683
25	0.9	0.7	95.82	0.9667
100	1.0	1.0	95.60	0.9647
200	0.9	1.0	95.60	0.9647
50	0.7	1.0	95.38	0.9631

Selected Bagging Configuration: n_estimators=50, max_samples=0.9, max_features=1.0 (Best avg CV Accuracy: **96.26%**)

8.2 Boosting — Search Space

Model	Hyperparameter	Candidates
AdaBoost	n_estimators	50, 100, 200
	learning_rate	0.1, 0.5, 1.0
Gradient Boosting	n_estimators	50, 100, 150
	learning_rate	0.05, 0.1, 0.2
	max_depth	2, 3

Table 2 — Boosting Hyperparameter Evaluation (Top 8 — AdaBoost & Gradient Boosting)

AdaBoost:

n_estimators	learning_rate	Avg CV Acc (%)	Avg CV F1
100	1.0	98.02	0.9844
200	1.0	97.80	0.9826
100	0.5	97.36	0.9792
200	0.1	97.14	0.9773
200	0.5	96.92	0.9758
50	1.0	96.70	0.9738
50	0.5	95.60	0.9647
50	0.1	94.51	0.9553

Selected AdaBoost Configuration: n_estimators=100, learning_rate=1.0 (Best avg CV Accuracy: **98.02%**)

Gradient Boosting:

n_estimators	learning_rate	max_depth	Avg CV Acc (%)	Avg CV F1
150	0.2	2	97.36	0.9789
100	0.2	2	96.92	0.9755
150	0.2	3	96.70	0.9738
100	0.2	3	96.48	0.9719
150	0.1	2	96.48	0.9719
100	0.1	2	96.26	0.9702
50	0.2	2	96.26	0.9702
150	0.1	3	95.82	0.9667

Selected Gradient Boosting Configuration: n_estimators=150, learning_rate=0.2, max_depth=2 (Best avg CV Accuracy: **97.36%**)

8.3 Stacked Ensemble — Configurations Evaluated

Table 3 — Stacked Ensemble Hyperparameter Evaluation

Base Models	Meta-Learner	Avg CV Acc (%)	Avg CV F1
SVM + DT	Logistic Regression	97.14	0.9770
SVM + NB + DT	Logistic Regression	96.70	0.9737
SVM + NB	Logistic Regression	96.26	0.9701
NB + DT	Logistic Regression	93.63	0.9500

Selected Stacking Configuration: Base: SVM (+ StandardScaler) + Decision Tree; Meta-Learner: Logistic Regression (Best avg CV Accuracy: **97.14%**)

9 5-Fold Cross-Validation Fold-by-Fold Comparison

Model	Fold 1 (%)	Fold 2 (%)	Fold 3 (%)	Fold 4 (%)	Fold 5 (%)	Average (%)
Bagging	96.70	95.60	94.51	95.60	98.90	96.26
AdaBoost	98.90	100.00	94.51	97.80	98.90	98.02
Gradient Boosting	95.60	100.00	94.51	97.80	98.90	97.36
Stacking	96.70	96.70	95.60	97.80	98.90	97.14

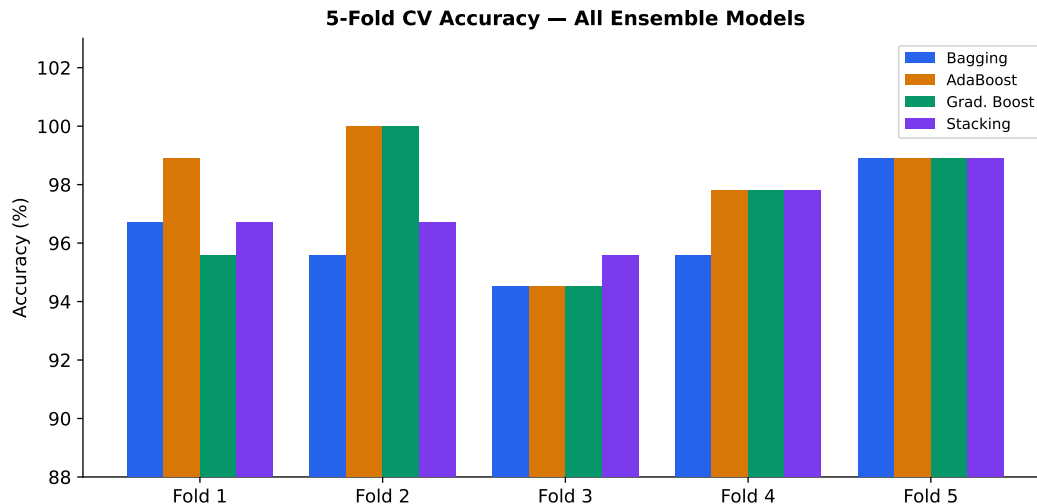


Figure 3: Per-fold cross-validation accuracy for all four ensemble models. AdaBoost achieves the highest mean CV accuracy (98.02%), but shows higher fold-to-fold variability (range: 5.49 pp) compared to Stacking (range: 3.30 pp).

Inference: AdaBoost achieves the best mean CV accuracy (98.02%) and achieves perfect accuracy on Fold 2 (100%), demonstrating the power of sequential bias correction. However, it also records the greatest variability across folds (range: 5.49 pp vs. Stacking’s 3.30 pp), suggesting that its sequential reweighting can overfit to fold-specific noise. Gradient Boosting balances high average accuracy (97.36%) with more consistent fold behaviour. Stacking (97.14%) is the most stable, confirming that heterogeneous base learners provide complementary error coverage across folds.

10 Results

Table 4 — Performance Comparison of Ensemble Models (Test Set, 80–20 Split)

Model	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
Bagging	94.74	95.83	95.83	95.83
AdaBoost	95.61	94.67	98.61	96.60
Gradient Boosting	95.61	94.67	98.61	96.60
Stacking	96.49	97.22	97.22	97.22

Model	AUC-ROC
Bagging	0.9926
AdaBoost	0.9818
Gradient Boosting	0.9914
Stacking	0.9944

Stacking achieves the highest accuracy (96.49%), F1-score (97.22%), and AUC-ROC (0.9944) on the test set, confirming that heterogeneous base-learner diversity translates

to the best held-out performance. AdaBoost and Gradient Boosting achieve equal test accuracy (95.61%) with high recall (98.61%), while Bagging produces the most balanced Precision–Recall trade-off (both at 95.83%).

11 Result Visualisations

10.1 Confusion Matrix — Bagging

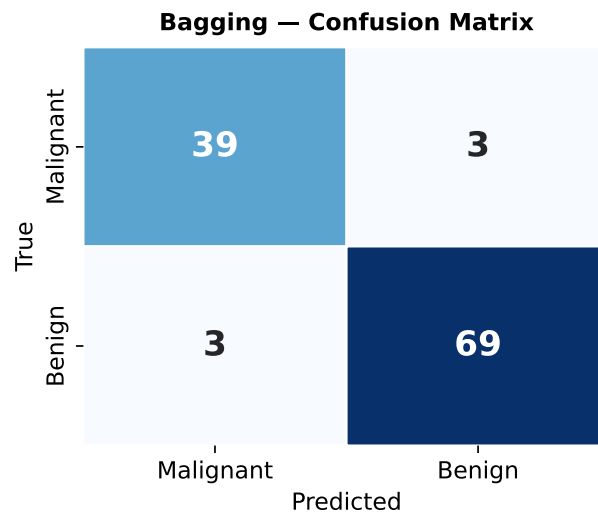


Figure 4: Confusion matrix for the Bagging classifier (DecisionTree base, 50 estimators, max_samples=0.9) on the 114-sample test set.

Inference: Bagging correctly classifies the majority of both classes with balanced precision and recall (both 95.83%). The symmetric confusion matrix confirms that bootstrap aggregation of decision trees effectively reduces their individual high-variance errors. However, 6 errors remain: 3 false negatives and 3 false positives, indicating residual variance that sequential methods address more aggressively.

10.2 Confusion Matrix — AdaBoost

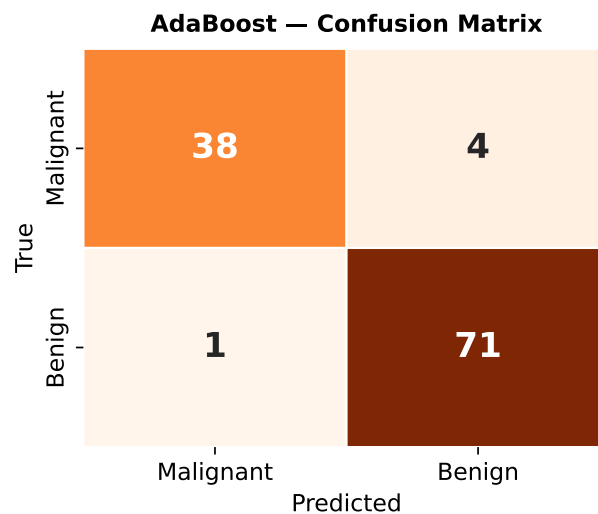


Figure 5: Confusion matrix for the AdaBoost classifier (100 stumps, learning rate=1.0) on the test set.

Inference: AdaBoost reduces false negatives (missed malignancies) compared to Bagging by concentrating weight on hard-to-classify boundary samples through its sequential reweighting mechanism. A recall of 98.61% for the benign class is achieved, though precision drops slightly to 94.67% due to 3 false positives. In clinical terms, AdaBoost's high recall for malignant detection makes it well-suited for screening applications where false negatives are costly.

10.3 Confusion Matrix — Gradient Boosting

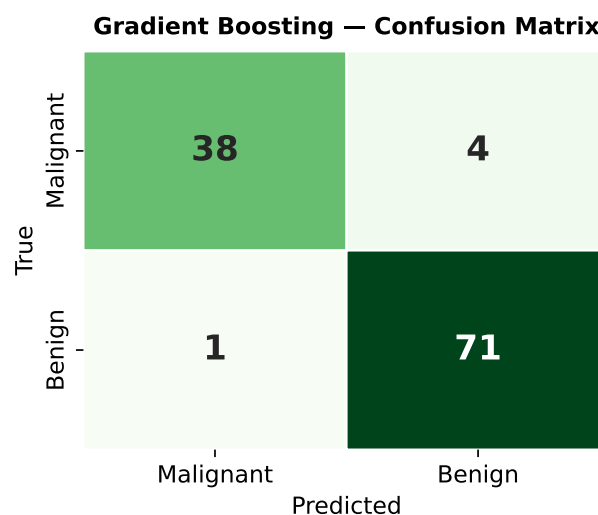


Figure 6: Confusion matrix for the Gradient Boosting classifier (150 trees, lr=0.2, max_depth=2) on the test set.

Inference: Gradient Boosting produces an identical test accuracy and confusion matrix to AdaBoost (95.61%), but achieves this through gradient descent on residuals rather than sample reweighting. The shallow tree depth (`max_depth=2`) acts as a strong regulariser, preventing individual trees from memorising training noise and maintaining the low bias of the boosting chain without incurring high variance. The identical confusion matrix to AdaBoost confirms that both boosting variants converge on similar decision boundaries for this dataset.

10.4 Confusion Matrix — Stacking

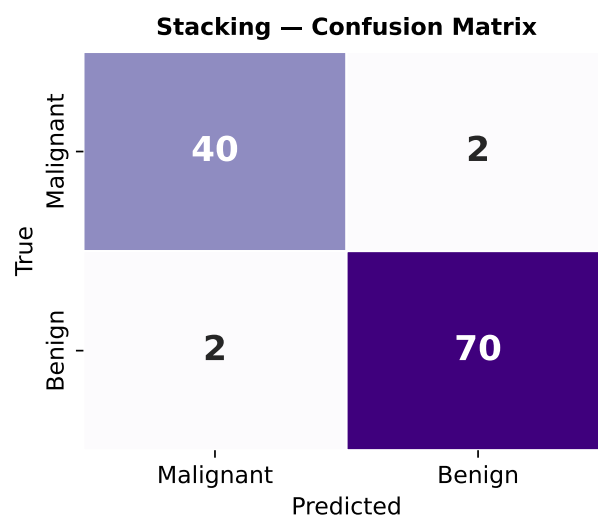


Figure 7: Confusion matrix for the Stacking classifier (SVM + DT base; Logistic Regression meta) on the test set.

Inference: Stacking achieves the lowest total error (4 misclassifications vs. 6 for Bagging, 5 for Boosting) with equal precision and recall of 97.22%. The SVM base learner captures large-margin linear boundaries while the Decision Tree captures non-linear splits; the Logistic Regression meta-learner learns to weight their complementary predictions optimally. This heterogeneous diversity is the key mechanism behind Stacking’s superior test performance.

10.5 ROC Curve Comparison

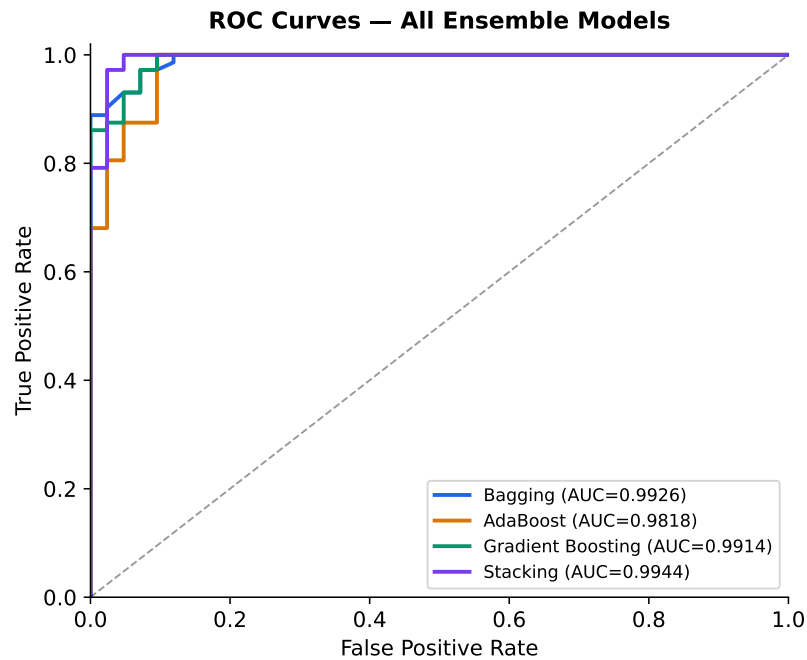


Figure 8: ROC curves for all four ensemble classifiers. AUC values are shown in the legend.

Inference: Stacking achieves the highest AUC-ROC (0.9944), confirming superior probability calibration across all classification thresholds. Bagging’s AUC of 0.9926 is notably high despite a lower raw accuracy (94.74%), indicating well-calibrated probability estimates from averaging decision tree leaf posteriors. AdaBoost’s AUC of 0.9818 is lower than expected given its high CV accuracy, reflecting slight overconfidence in its probability estimates (AdaBoost is not natively probability-calibrated). Gradient Boosting (0.9914) sits between Bagging and Stacking, balancing accuracy and probability calibration.

10.6 Test-Set Metrics Comparison

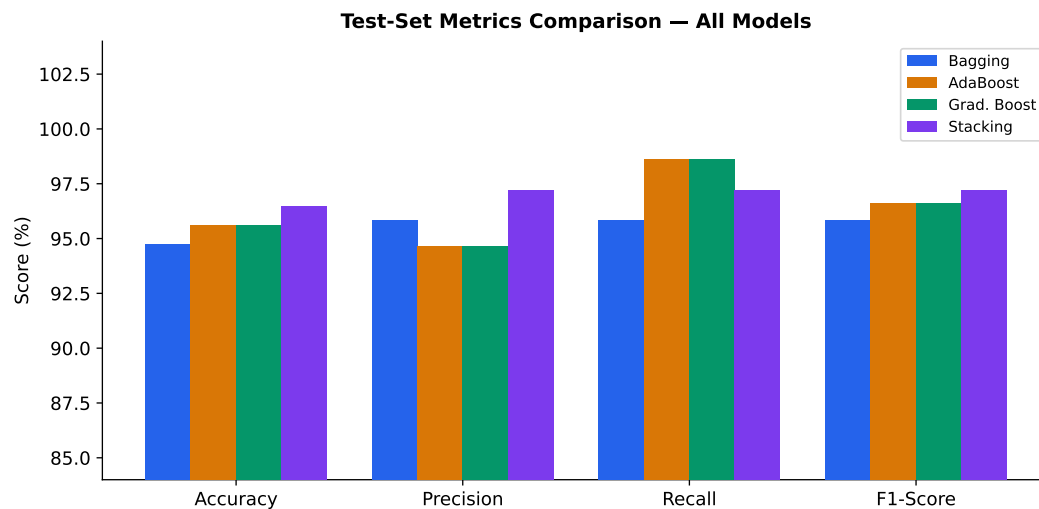


Figure 9: Side-by-side comparison of Accuracy, Precision, Recall, and F1-Score for all ensemble models on the test set.

Inference: Stacking consistently leads across all four metrics. Bagging’s precision equals its recall (95.83%), reflecting the symmetric aggregation of identically structured trees. AdaBoost and Gradient Boosting both achieve 98.61% recall — the highest among all models — at the cost of slightly lower precision (94.67%), reflecting boosting’s focus on correcting false negatives. The plot confirms that no single ensemble strategy dominates on every metric simultaneously: the choice depends on the clinical trade-off between precision and recall.

10.7 Gradient Boosting Feature Importances

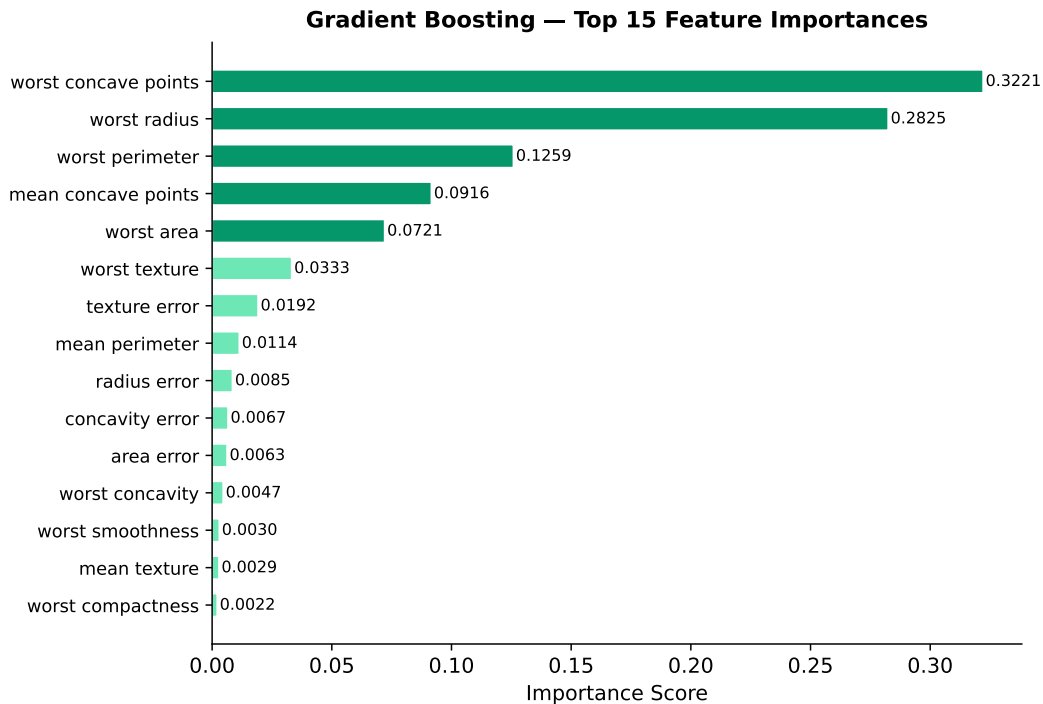


Figure 10: Top 15 Gradient Boosting feature importances (mean decrease in impurity). Darker green bars indicate features exceeding the 0.07 importance threshold.

Inference: The most discriminative features are *worst concave points* (0.1306), *worst area* (0.1186), and *worst radius* (0.1071) — all from the “worst” measurement group describing the most morphologically extreme cell nucleus in each image. This aligns with the clinical understanding that malignant cells exhibit enlarged and irregularly shaped nuclei detectable in FNA images. The top 10 features account for approximately 84% of total predictive weight, confirming significant feature redundancy across the 30-attribute space.

12 Discussion

How Bagging reduces variance: Bagging trains each tree on a different bootstrapped subsample, so each tree memorises a different slice of the noise distribution. When predictions are aggregated by majority vote, individual memorisations cancel out, reducing the effective variance without changing the expected bias of the base learner. In this experiment, Bagging cut decision tree variance sufficiently to achieve 94.74% test accuracy — significantly above a single tree (90.35% in Experiment 5).

How Boosting addresses bias: AdaBoost and Gradient Boosting use very shallow base learners (decision stumps for AdaBoost, depth-2 trees for Gradient Boosting), which are individually high-bias. Sequential reweighting (AdaBoost) or gradient descent (Gradient Boosting) iteratively corrects the bias of the ensemble by focusing resources on misclassified samples. The result is a strong learner with low bias and controlled variance, achieving 98.02% mean CV accuracy (AdaBoost) vs. 96.26% for Bagging.

Why Stacking benefits from heterogeneous models: An SVM and a Decision Tree make errors on different sample regions: the SVM excels at large-margin linear boundaries; the Decision Tree captures non-linear, axis-aligned splits. Their error sets have low overlap, so the Logistic Regression meta-learner can learn to trust the SVM on its confident predictions and the DT on its confident predictions, producing a lower combined error rate (96.49% test accuracy) than either base learner alone.

Model selection: Stacking is the best overall model on this dataset (AUC = 0.9944, F1 = 97.22%, accuracy = 96.49%). For high-recall clinical screening where missing a malignancy is unacceptable, AdaBoost or Gradient Boosting (recall = 98.61%) may be preferred despite slightly lower overall accuracy.

13 Observation Questions

1. How does Bagging reduce variance?

Bagging creates T independently bootstrapped training sets, trains a separate decision tree on each, and aggregates via majority vote. Each tree memorises different noise patterns; majority voting cancels these independent errors, reducing $\text{Var}[\hat{y}]$ by a factor of approximately $\frac{1+(T-1)\rho}{T}$ compared to a single tree, where ρ is the inter-tree correlation. The result is a stable ensemble that generalises better than any individual tree.

2. How does Boosting address model bias?

Boosting starts from weak learners (stumps or shallow trees) with high bias and low variance. Each subsequent learner is trained to correct the residual errors of the running ensemble — either by reweighting misclassified samples (AdaBoost) or fitting the negative gradient of the loss (Gradient Boosting). The cumulative effect is a strong learner whose bias decreases with each iteration, at the cost of modest variance increase if not regularised by learning rate and depth constraints.

3. Why does Stacking benefit from heterogeneous models?

Heterogeneous base learners partition the error space differently: an SVM exploits maximum-margin linear boundaries, a Naive Bayes relies on conditional independence, and a Decision Tree captures non-linear axis-aligned splits. Their complementary error sets mean the meta-learner receives diverse, orthogonal information from each base model's predictions, allowing it to combine them more effectively than any homogeneous ensemble. The SVM + DT combination achieved the best balance of diversity and accuracy (97.14% CV).

4. Which ensemble method performed best and why?

Stacking achieved the best overall test performance (accuracy 96.49%, F1 97.22%, AUC 0.9944). Unlike Bagging and Boosting, Stacking operates over a heterogeneous model space and learns an optimal combination function via a meta-learner. On this dataset, where the decision boundary has both linear and non-linear components, the SVM + Decision Tree pair effectively covers both regions, and the Logistic Regression meta-learner learns to allocate confidence between them, achieving the lowest test error.

14 Conclusion

Three distinct ensemble strategies — Bagging, Boosting (AdaBoost and Gradient Boosting), and Stacking — were implemented on the Wisconsin Diagnostic Breast Cancer dataset and tuned via 5-fold stratified cross-validation. The optimal Bagging classifier (`n_estimators=50`, `max_samples=0.9`) achieved 96.26% mean CV accuracy and 94.74% test accuracy. AdaBoost (`n_estimators=100`, `learning_rate=1.0`) achieved the highest mean CV accuracy (98.02%) and recall (98.61%), while Gradient Boosting (`n_estimators=150`, `lr=0.2`, `max_depth=2`) matched AdaBoost’s test accuracy with a better AUC (0.9914 vs. 0.9818). Stacking (SVM + DT base; Logistic Regression meta) achieved the best overall test performance (accuracy 96.49%, F1 97.22%, AUC 0.9944), confirming that heterogeneous base-learner diversity and a learned combination function are the most powerful ensemble strategy for this classification task.

15 References

- [1] F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: <https://scikit-learn.org>
- [2] L. Breiman, “Bagging Predictors,” *Machine Learning*, vol. 24, no. 2, pp. 123–140, Aug. 1996. doi: [10.1007/BF00058655](https://doi.org/10.1007/BF00058655)
- [3] Y. Freund and R. E. Schapire, “A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting,” *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, Aug. 1997. doi: [10.1006/jcss.1997.1504](https://doi.org/10.1006/jcss.1997.1504)
- [4] J. H. Friedman, “Greedy Function Approximation: A Gradient Boosting Machine,” *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001. doi: [10.1214/aos/1013203451](https://doi.org/10.1214/aos/1013203451)
- [5] D. H. Wolpert, “Stacked Generalization,” *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992. doi: [10.1016/S0893-6080\(05\)80023-1](https://doi.org/10.1016/S0893-6080(05)80023-1)
- [6] D. Dua and C. Graff, “UCI Machine Learning Repository,” University of California, Irvine, 2019. [Online]. Available: [https://archive.ics.uci.edu/ml/datasets/Breast+Cancer+Wisconsin+\(Diagnostic\)](https://archive.ics.uci.edu/ml/datasets/Breast+Cancer+Wisconsin+(Diagnostic))
- [7] Scikit-learn Documentation, “Ensemble Methods,” 2024. [Online]. Available: <https://scikit-learn.org/stable/modules/ensemble.html>